Day 8 Evidence Workbooklet

Expected-vs-Actual Debugging and Test Design

Engineering practice → observed branch outcome → expected branch outcome → likely cause → regression test

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/34		Mastery check	secure / developing / needs support	

The sensor-kit checkout decision logic is now being tested. Some branch outcomes look safe for one kit but fail for another. Today the focus is debugging: separate the observed behavior from the expected behavior, identify a likely cause, choose a minimal fix, and keep a test that would catch the bug again.

Engineering task	Use a repair log to decide whether the checkout decision logic is trustworthy before the lab uses it.
Computational move	Compare expected and actual behavior, connect the mismatch to a code cause, and design tests that cover important branch outcomes.
Python focus	expected branch outcome, actual branch outcome, condition order, Boolean release condition, minimal fix, boundary and regression tests.
Evidence to keep	symptom, expected branch outcome, actual branch outcome, likely cause, minimal repair, test case, and support category.

Day 8 working rules

1. A symptom says what went wrong.
2. Expected behavior says what should have happened.
3. Actual behavior says what the code did.
4. A likely cause points to a condition or order problem.
5. A regression test keeps a known bug from quietly returning.

1 Separate symptom, cause, and repair

recognize debug

The checkout desk finds that some kits are assigned incorrectly. A good debug note separates what happened, what should have happened, and what line likely caused it.

```
charge_percent = 82
calibration_label = "expired"
borrower_initials = "KM"

if charge_percent >= 80:
    action = "release kit"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "hold for review"
```

Pts.	Prompt	My evidence
1	What branch outcome will the current code assign?	
1	What branch outcome should the kit receive if calibration is expired?	
1	Which line lets the kit release too early?	
1	Is the first problem a syntax error or a logic error?	

2 Build an expected-vs-actual table

trace debug

Use the same code. Complete the table and name the likely cause.

Pts.	Case	Expected branch outcome	Actual branch outcome	Likely cause
2	charge 82, expired calibration	calibration bench		
2	charge 76, current calibration	charge station		
2	charge 90, current calibration, blank initials	borrower log needed		
2	Explain which case best exposes the bug.			

3 Design tests before trusting a repair

test explain

A repair is not trustworthy just because it works for one case. Pick cases that cover the branch outcomes.

Pts.	Prompt	My test evidence
2	Give a test case that should produce `"charge station"`.	
2	Give a test case that should produce `"calibration bench"`.	
2	Give a test case that should produce `"borrower log needed"`.	
2	Give a test case that should produce `"release kit"`.	

Regression-test idea

A regression test is a small case you keep because it catches a bug that already happened once. If the bug comes back, that test should fail again.

4 Choose a minimal fix

implement debug

One safe repair is to require all release evidence before assigning `"release kit"`. Complete the condition.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    action = "release kit"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "hold for review"
```

Pts.	Prompt	My response
2	Why does the release condition need three evidence checks?	
2	Which branch outcome will the expired-calibration case now receive?	
2	What risk remains if damage evidence is added later but not checked first?	
2	Name one test you would rerun after this repair.	

5 Write a repair log

debug test explain

Log field	My repair evidence
Symptom	
Expected behavior	
Likely cause	
Minimal fix	
Regression test	

6 Transfer and support plan

transfer explain

Pts.	Debugging note
4	Explain why expected-vs-actual evidence is stronger than saying "the code is wrong." Use one branch outcome from the sensor-kit checkout example.
2	Circle the support plan you would use next: symptom / expected branch outcome / actual branch outcome / cause / test case / minimal fix. Name one next action: _____

Bridge to Exam 1 and Day 10

Day 9 is Exam 1. Use expected-vs-actual evidence, minimal repair, boundary tests, and support-plan notes as Exam 1 preparation. Day 10 returns to branch maps and test evidence after the exam and bridges into repeated cases and loops.

7 Lab bridge: use visible checks as debugging evidence

VIP 18.8 unscored

Scope boundary: keep the interface fixed

Do not rewrite the `def` line, parameter names, or exact output labels. Debug only the order of the indented decision rules. This page adds no points; the workbook score remains unchanged.

The provided wrapper below has the right inputs and output labels, but the rules are in the wrong order.

```
def badge_sheet_plan(group_count, badges_per_group):
    total_badges = group_count * badges_per_group
    if total_badges <= 16:
        return "two sheets"
    elif total_badges <= 8:
        return "one sheet"
    else:
        return "print batch"
```

Total	Expected	Actual from wrong order	Evidence / likely cause
8	one sheet		
9	two sheets		
17	print batch		

Minimal repair	
Regression tests to keep	

Visible-check recovery loop

Run the editable file cell, run the visible checks, compare expected with observed, revise one rule, and rerun both cells. Do not change the setup or check cells to make a failure disappear.

Bridge Day 8 VIP Evidence Handoff

STUDENT COPY • Contract 07a04826c975 • Accessible v5 successor

Why engineers use this page

Use expected-versus-actual evidence on the current sample and transfer cells instead of debugging an obsolete wrapper.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: comparison expression, boolean operator, if elif else, abs call, counter update
Carried authoring tools: assignment, print call, numeric literal, arithmetic expression, input call, int conversion, float conversion, f string
Supplied-code exposure only: for loop, list traversal
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.13	Compare With Tolerance	`18-13-work / 18-13-transfer / 18-13-cold`	comparison expression, f string, float conversion, input call
18.14	Count High Readings	`18-14-work / 18-14-transfer / 18-14-cold`	arithmetic expression, comparison expression, counter update, f string, if elif else
18.15	Lamination Fee	`18-15-work / 18-15-transfer / 18-15-cold`	comparison expression, f string, if elif else, input call, int conversion
18.16	Temperature Status	`18-16-work / 18-16-transfer / 18-16-cold`	comparison expression, f string, float conversion, if elif else, input call
18.17	Pickup Window	`18-17-work / 18-17-transfer / 18-17-cold`	boolean operator, comparison expression, f string, if elif else, input call, int conversion

Readiness evidence

1. Choose one sample or transfer case and record expected result, actual result, likely cause, and the smallest justified repair.

2. Name one boundary pair that differs by only one unit or one small measurement change.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	
Observed Result	
Revision Reason	
Engineering Meaning	
Units Or Not Applicable	
Assumption	
Model Limit	
Next Test	
Help Trigger	
Minutes Used	

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.